.UBAfiuba
FACULTAD DE INGENIERÍA

Base de Datos

Curso Merlino

Trabajo Práctico 1
Patrones de Accesos de Diseño

6 de mayo de 2026

Mateo Sparta
Padron: 109418
DNI: 42624942

Jose Zerpa
Padron: 110698
DNI: 95980469

Facundo Sorasio
Padron: 108882
DNI: 43656024

Rodrigo Vargas
Padron: 97076
DNI: 94908785

Tomas Ruiz
Padron: 106904
DNI: 42689730

1. Prompts

1.1. Primer Prompt

Actúa como un arquitecto de software con experiencia académica y profesional en bases de datos relacionales y no relacionales. Explicá técnicamente los patrones y estructura en exactamente cuatro secciones, en el siguiente orden:

1.1.1. Sección 1: Introducción

Redacta una introducción académica, tecnica y precisa de entre 3 y 4 párrafos. Debe cumplir con los siguientes objetivos:

1. Presentar el tema de los patrones de acceso y diseño a bases de datos como un campo consolidado de la ingeniería de software.
2. Definí brevemente que es una DB ralacional y una no relacional.
3. Distinguir brevemente entre bases de datos relacionales y no relacionales como paradigmas complementarios que coexisten en la industria moderna.
4. Argumentar por qué el conocimiento de estos patrones es más relevante que nunca en un contexto donde la generación de código se delega crecientemente a modelos de lenguaje (LLMs). Si la implementación de ellas puede ser automatizada, el valor diferencial del profesional recae en las decisiones de diseño y arquitectura que ningún modelo puede tomar sin una dirección.

1.1.2. Sección 2: Cuadro comparativo

Defini una tabla con los principales patrones (ActiveRecord, Data Mapper, Repository, Unit of Work ,DAO, entre otros.), distribuidos equitativamente entre relacionales y no relacionales. Agrúpalos en una table que contenga estos 6 datos, en este orden: 1. Nombre del patrón 2. Categoría (acceso, diseño de datos, mixto) 3. Tipo de BD (relacional, no relacional y/o subtipo) 4. Ventajas 5. Desventajas 6. Implementaciones ideales (frameworks, ORMs o motores que lo adoptan, etc.)

1.1.3. Sección 3: Glosario y referencias técnicas

Inclui un apartado que desarrolle los conceptos técnicos necesarios para interpretar correctamente el cuadro. La idea es que el cuadro pueda mantenerse resumido porque el lector tiene aquí el respaldo con información mas detallada. Usa registro formal y vocabulario técnico preciso, sin caer en generacion artificial de contendio ni en densidad innecesaria.

1.2. Ultimo Prompt

en base a la naturaleza del trabajo practico, escribe una conclusion sobre el tema. Ademas, aporta sobre los patrones mas usados en el dia a dia de un ingeniero de software, y tambien de aquellos que son mas valorados en el tiempo de la IA. Debe ser de al menos 3 parrafos o 250 palabras aproximadamente.

Sección 1: Introducción

1. Los Patrones de Acceso como Pilar de la Ingeniería

El diseño de patrones de acceso y persistencia de datos constituye uno de los pilares fundamentales y más consolidados de la ingeniería de software contemporánea. No se trata simplemente de un conjunto de reglas para almacenar información, sino de un cuerpo de conocimiento arquitectónico que permite desacoplar la lógica de negocio de los mecanismos de almacenamiento. A lo largo de las últimas décadas, estos patrones han evolucionado desde simples abstracciones de archivos hasta complejas capas de mediación que garantizan la integridad, escalabilidad y mantenibilidad de sistemas críticos a nivel global.

2. Paradigmas de Almacenamiento: RDBMS y NoSQL

Para comprender este ecosistema, es imperativo distinguir entre los dos paradigmas dominantes. Una **Base de Datos Relacional (RDBMS)** se fundamenta en el modelo relacional de Codd, organizando los datos en tablas con esquemas rígidos y garantizando propiedades **ACID** (*Atomicity, Consistency, Isolation, Durability*) mediante el uso de SQL. Por otro lado, una **Base de Datos No Relacional (NoSQL)** ofrece modelos de datos más flexibles (documentos, grafos, clave-valor o columnares), priorizando generalmente el escalamiento horizontal y la disponibilidad sobre la consistencia inmediata, bajo los preceptos del **teorema CAP**.

3. Complementariedad y Persistencia Políglota

En la industria moderna, estos paradigmas no compiten en un juego de suma cero, sino que coexisten como soluciones complementarias dentro de arquitecturas de **persistencia políglota**. Mientras que los motores relacionales siguen siendo el estándar de oro para transacciones financieras y datos estructurados complejos, los sistemas NoSQL permiten gestionar volúmenes masivos de datos no estructurados y requisitos de baja latencia. La decisión arquitectónica no reside en elegir cuál es mejor, sino en determinar qué patrón de acceso y qué motor se alinean de forma óptima con los atributos de calidad del sistema en desarrollo.

4. El Rol del Arquitecto frente a la Automatización por LLMs

La arquitectura de persistencia ha dejado de ser una cuestión de .ªlmacenamiento"para convertirse en una disciplina de gestión de la complejidad. Como hemos analizado, la elección entre un patrón ActiveRecord o un Data Mapper, o la implementación de una capa de Repository, no responde a modas tecnológicas, sino a una evaluación técnica de los requisitos de acoplamiento, testeabilidad y escalabilidad del sistema. En la práctica académica y profesional, la conclusión es clara: una base de datos es un detalle de implementación; lo que realmente define la longevidad de un software es la solidez de sus contratos de acceso a datos.

Patrones de uso cotidiano: El estándar de la industria En el día a día de un ingeniero de software, el patrón Repository se ha consolidado como el estándar de facto. Su ubicuidad se debe a que permite tratar el almacenamiento como una colección en memoria, facilitando enormemente las pruebas unitarias mediante el uso de mocks. Junto a este, el patrón DTO (Data Transfer Object) es indispensable para mover datos entre capas sin exponer las entidades internas del dominio o los esquemas de la base de datos, evitando que cambios en la infraestructura rompan"la interfaz de usuario o las APIs externas.

2. Cuadro comparativo de patrones

Nombre	Categoría	Tipo de BD	Ventajas	Desventajas	Implementaciones
ActiveRecord	Acceso	Relacional	• Desarrollo ágil • Lógica y datos juntos • Intuitivo para CRUD	• Acoplamiento a la DB • Difícil de testear • Viola SRP	Ruby on Rails, Eloquent
Data Mapper	Acceso	Relacional	• Desacoplamiento total • Soporta esquemas complejos • Alta testabilidad	• Curva de aprendizaje • Mucho código inicial • Configuración densa	Hibernate, Doctrine
Repository	Acceso	Rel/Mixto	• Visión de colección • Sustitución de fuente • Centraliza consultas	• Puede ocultar el SQL • Capa extra innecesaria	Spring Data, MediatR
Unit of Work	Mixto	Relacional	• Transacciones atómicas • Optimiza llamadas • Gestión de estados	• Complejidad lógica • Dificultad manual	Entity Framework
Identity Map	Mixto	Relacional	• Evita duplicación • Consistencia en memoria	• Alto consumo RAM • Gestión de caché	JPA, NHibernate
Table Gateway	Acceso	Relacional	• SQL centralizado • Estructura simple	• Poco flexible • Duplica lógica	Zend DB, .NET DS
Embedded Doc	Diseño	NoSQL (Doc)	• Lecturas rápidas • Sin joins costosos • Localidad de datos	• Límite de tamaño • Datos redundantes	MongoDB, CouchDB
Linking	Diseño	NoSQL (Doc)	• Sin redundancia • Actualización fácil	• Varias consultas • Mayor latencia	MongoDB DBRef
Bucket	Diseño	NoSQL (Series)	• Optimiza series • Bajo índice masivo	• Rangos complejos • Requiere agregación	InfluxDB, MongoDB
Materialized V.	Acceso	NoSQL (KV)	• Lectura instantánea • Escala extrema	• Consistencia eventual • Costo de storage	DynamoDB, Redis
Wide Column	Mixto	NoSQL (Col)	• Consultas masivas • Alta compresión	• Diseño rígido • Difícil de cambiar	Cassandra, ScyllaDB
Saga	Acceso	NoSQL (Dist)	• Consistencia eventual • Tolerancia a fallos	• Lógica compleja • Depuración difícil	Axon Framework

3. Glosario y referencias técnicas

Para una interpretación rigurosa de la arquitectura de datos, es necesario precisar los siguientes conceptos técnicos:

- **Impedance Mismatch (Desajuste de Impedancia):** Se refiere a la diferencia conceptual entre el modelo de objetos (programación) y el modelo relacional (tablas). Los patrones como Data Mapper buscan mitigar esta brecha.
- **ORM (Object-Relational Mapping):** Técnica de programación para convertir datos entre sistemas de tipos incompatibles mediante el uso de objetos.
- **ACID vs. BASE:** Mientras que ACID busca consistencia estricta, el modelo BASE (Basically Available, Soft state, Eventual consistency) es el pilar de los sistemas NoSQL distribuidos, donde la consistencia se alcanza eventualmente para favorecer la disponibilidad.
- **Abstracción de Persistencia:** Capa de software que oculta los detalles de implementación del almacenamiento. Permite que la lógica de negocio opere con abstracciones (objetos/entidades) en lugar de primitivas de base de datos (filas/documentos).
- **Escalamiento Horizontal (Sharding):** Técnica común en NoSQL que consiste en distribuir los datos en múltiples servidores. A diferencia del escalamiento vertical (más hardware en un solo equipo), el horizontal requiere patrones de diseño que soporten la partición de datos.
- **Inyección de Dependencias (DI):** Patrón de diseño fundamental para implementar Repositories y DAOs, permitiendo que la capa de negocio reciba la implementación de persistencia de forma externa, facilitando las pruebas unitarias y el intercambio de motores.

4. Conclusión y criterios de diseño

La arquitectura de persistencia ha evolucionado de ser una tarea rutinaria de almacenamiento a una **disciplina crítica de gestión de la complejidad**. La conclusión técnica es que el motor de base de datos debe ser tratado como un simple *detalle de implementación*; lo que garantiza la longevidad del software es la solidez de sus contratos de acceso y la abstracción de sus capas. En la actual era de la automatización, el valor del ingeniero no reside en escribir código, sino en la toma de decisiones sobre la **estrategia arquitectónica**.

En la práctica cotidiana, el patrón *Repository* se mantiene como el **estándar de oro**, pues permite tratar la persistencia como una colección lógica en memoria, facilitando el testeo unitario y el desacoplamiento total del motor. Complementariamente, los **DTOs (Data Transfer Objects)** resultan esenciales para proteger la integridad del dominio frente a fluctuaciones externas en la infraestructura.

Frente al auge de los LLMs, donde la generación de código táctico es instantánea, los patrones más valorados son aquellos que potencian la **adaptabilidad**. La *Arquitectura Hexagonal* (Puertos y Adaptadores) se vuelve indispensable para aislar la lógica de negocio y permitir intercambiar tecnologías de persistencia sin fricción. Asimismo, patrones de resiliencia como el *Circuit Breaker* cobran una relevancia inédita al integrar servicios de inteligencia artificial externos que presentan latencias variables.

En definitiva, mientras la IA asume la implementación mecánica de consultas y mapeos, el arquitecto debe asegurar que el sistema sea **flexible y fácil de evolucionar**. La madurez profesional se mide hoy por la capacidad de diseñar estructuras que trasciendan la obsolescencia tecnológica.